



Universidade Federal
do Rio de Janeiro

Escola Politécnica

EEL891 — Introdução ao Aprendizado de Máquina

Trabalho 1

Classificação de Inadimplência de Crédito

Classificador binário para apoio à decisão

Aluno: Luiz Felipe Píccoli Cavalini

Ferramentas: Python 3.12 (scikit-learn, LightGBM,
XGBoost, CatBoost, Optuna)

Rio de Janeiro, Julho de 2026

Sumário

1	Introdução	1
2	Análise Exploratória dos Dados	1
2.1	Estrutura e distribuição do <i>target</i>	1
2.2	Valores ausentes	1
2.3	Colunas descartadas	1
2.4	Correlações e visualizações	1
3	Pré-processamento	2
3.1	Variáveis textuais	2
3.2	Imputação de nulos	2
4	Engenharia de <i>Features</i>	2
5	Codificação de Variáveis Categóricas	3
5.1	Label Encoding	3
5.2	Target Encoding	3
5.2.1	Abordagens testadas e descartadas	3
6	Modelagem	4
6.1	Validação cruzada	4
6.2	Modelos base	4
7	Otimização de Hiperparâmetros	4
8	<i>Ensemble</i> e Submissão	5
8.1	Voting Ensemble	5
8.2	Blend multi-seed	5
8.3	Abordagens de ensemble testadas e descartadas	6
9	Resultados	6
10	Importância das <i>Features</i>	6
11	Conclusões	8
	Referências	9

1 Introdução

O presente trabalho consiste na construção de um classificador binário para apoio à decisão de aprovação de crédito. A partir de dados históricos de 20.000 solicitações de crédito, acompanhadas do desfecho (adimplência ou inadimplência), o objetivo é prever o comportamento de 5.000 novas solicitações cujos desfechos são desconhecidos.

A métrica de avaliação é a **acurácia**, e as previsões são submetidas na plataforma Kaggle, que compara automaticamente com o gabarito oculto.

O pipeline adotado segue as etapas clássicas: análise exploratória, pré-processamento, engenharia de *features*, modelagem com múltiplos algoritmos, otimização de hiperparâmetros e construção de *ensembles*. As ferramentas utilizadas foram Python 3.12 com *pandas* [10], *scikit-learn* [1], *LightGBM* [2], *XGBoost* [3], *CatBoost* [4] e *Optuna* [5].

2 Análise Exploratória dos Dados

2.1 Estrutura e distribuição do *target*

O conjunto de treinamento contém 20.000 amostras e 41 colunas. A variável-alvo *inadimplente* é **perfeitamente balanceada**: 10.000 bons pagadores e 10.000 inadimplentes (50%/50%). Isso dispensa técnicas de balanceamento como SMOTE ou ajuste de `class_weight`.

2.2 Valores ausentes

Seis colunas apresentam valores nulos. As mais afetadas são `grau_instrucao_companheiro` (64,3%) e `profissao_companheiro` (57,6%), cujos altos percentuais indicam provável ausência de companheiro — o que motivou a criação de uma *feature* binária `tem_companheiro`.

2.3 Colunas descartadas

Quatro colunas foram removidas: `grau_instrucao` (100% zeros), `possui_telefone_celular` (100% “N”), `qtde_contas_bancarias_especiais` (idêntica a outra coluna) e `local_onde_trabalha` (idêntica a `local_onde_reside` em 100% dos registros).

2.4 Correlações e visualizações

As correlações lineares com o *target* são todas muito fracas (máximo $|r| = 0,12$ para *idade*), indicando que modelos lineares teriam desempenho limitado e que modelos não-lineares baseados em árvores [6, 7] seriam mais adequados.

A Figura 1 apresenta a distribuição das variáveis mais relevantes, segmentadas por classe. Observa-se que nenhuma variável isolada separa bem as classes, confirmando

a necessidade de combinações não-lineares.

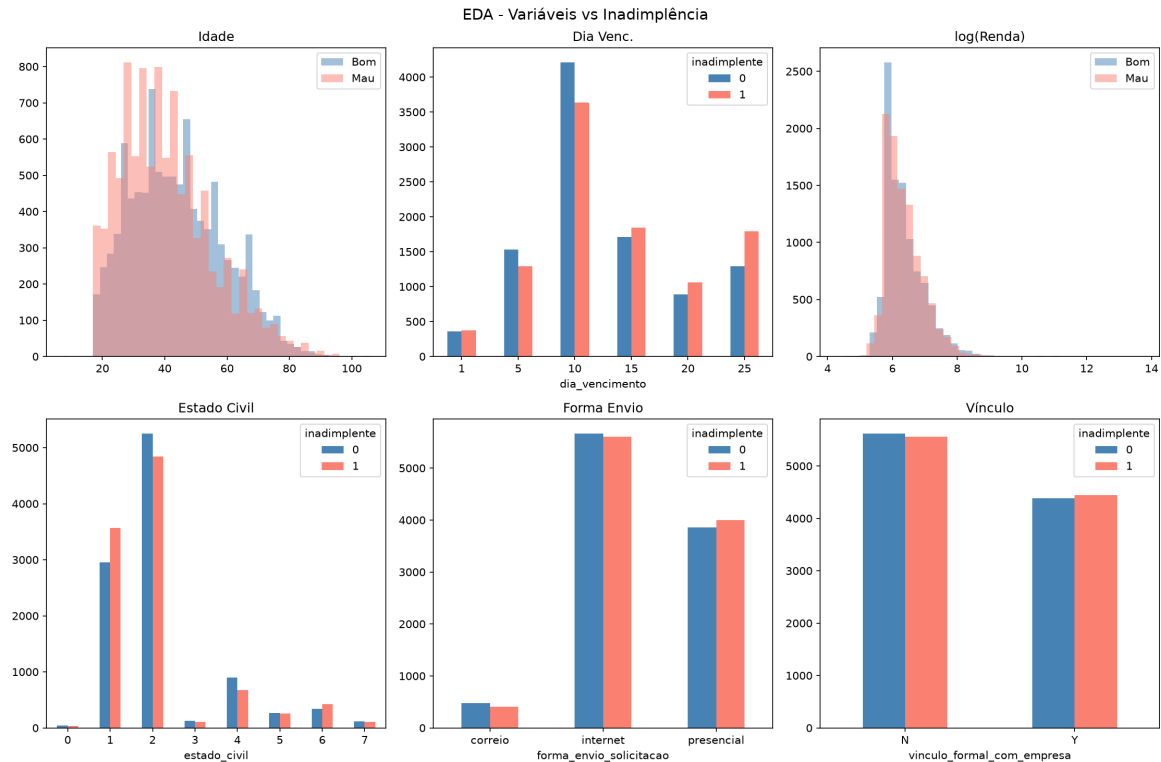


Figura 1: Distribuição das principais variáveis por classe (azul = bom pagador, vermelho = inadimplente).

3 Pré-processamento

3.1 Variáveis textuais

Valores em branco e códigos inválidos (“XX”) foram substituídos por categorias explícitas. As colunas Y/N foram convertidas para 0/1.

3.2 Imputação de nulos

A estratégia variou conforme o tipo: mediana para `meses_na_residencia`, moda para `tipo_residencia`, e valor sentinela -1 para `profissao`, `ocupacao` e as variáveis de companheiro (indicando “não informado”).

4 Engenharia de *Features*

Foram criadas 14 novas variáveis a partir dos atributos originais. A escolha de cada uma foi guiada pelo conhecimento do domínio de crédito e pela análise exploratória:

- **Financeiras:** `renda_total` (soma de renda regular e extra, capturando a capacidade de pagamento completa), `tem_renda_extra` (flag: fontes adicionais de renda

indicam maior estabilidade), `razao_patrimonio_renda` (patrimônio dividido pela renda — um patrimônio alto relativo à renda sugere solidez financeira acumulada).

- **Cartões:** `qtde_cartoes` e `tem_algum_cartao` — a posse de cartões indica acesso prévio a crédito e histórico no sistema financeiro.
- **Geográficas:** `reside_mesmo_estado` e `trabalha_mesmo_estado` — estabilidade geográfica pode correlacionar com estabilidade financeira.
- **Contato:** `qtde_telefones` — mais pontos de contato sugerem maior rastreabilidade do solicitante.
- **Transformações:** `faixa_etaria` (discretização da idade em 6 faixas, permitindo ao modelo capturar efeitos não-monotônicos), `log_renda`, `log_patrimonio` e `log_renda_total` (reduzir a assimetria causada por *outliers* extremos, como rendas de até R\$ 959.000).
- **Interações:** `idade_x_renda` (uma pessoa jovem com renda alta tem perfil diferente de uma idosa com a mesma renda — a interação captura esse efeito) e `vinculo_x_renda` (renda com vínculo formal tem significado diferente de renda informal de mesmo valor).

5 Codificação de Variáveis Categóricas

5.1 Label Encoding

Aplicado a `sexo` (3 categorias) e `forma_envio_solicitacao` (5 categorias).

5.2 Target Encoding

Para variáveis de alta cardinalidade (`estado_onde_nasceu`, `estado_onde_reside`, `estado_onde_trabalha`, `codigo_area_telefone_residencial`, `codigo_area_telefone_trabalho`), cada categoria foi substituída pela proporção média de inadimplentes, calculada **exclusivamente no conjunto de treinamento**, seguindo a técnica descrita por Micci-Barreca [8].

5.2.1 Abordagens testadas e descartadas

Durante o desenvolvimento, foram testadas variações de encoding:

- **Target encoding expandido:** estender o *target encoding* para todas as variáveis categóricas numéricas (`profissao`, `ocupacao`, `dia_vencimento`, etc.). A validação cruzada indicou um ganho expressivo ($0,604 \rightarrow 0,636$), porém ao submeter ao Kaggle a acurácia **caiu para 0,589**. A causa foi identificada como ***data leakage***: o encoding é calculado com informação do *target* no treino inteiro, incluindo os folds de validação, inflando artificialmente a CV.

- **Target encoding out-of-fold:** tentou-se corrigir o *leakage* calculando o encoding em cada fold separadamente. A CV ficou mais honesta (0,609), mas o Kaggle ainda retornou 0,594 — confirmando que essas variáveis, com muitas categorias raras, não se beneficiam dessa técnica.
- **Encoding ordinal puro** (sem *target encoding*): piorou para 0,587, confirmando a utilidade do *target encoding* para as 5 variáveis de estado e telefone.

Conclusão: o *target encoding* foi mantido apenas para as 5 variáveis de alta cardinalidade textuais, que demonstraram consistência entre CV e Kaggle.

6 Modelagem

6.1 Validação cruzada

Todos os modelos foram avaliados com **5-Fold Stratified Cross-Validation**, mantendo a proporção 50/50 do *target* em cada fold.

6.2 Modelos base

A Tabela 1 apresenta os resultados dos modelos com hiperparâmetros padrão.

Tabela 1: Acurácia CV dos modelos base

Modelo	Acurácia CV	Desvio
Regressão Logística	0,5638	0,0075
Random Forest [7]	0,6006	0,0090
LightGBM [2]	0,5993	0,0097
XGBoost [3]	0,6006	0,0111
CatBoost [4]	0,6082	0,0111

A Regressão Logística ficou abaixo dos demais, confirmando a necessidade de modelos não-lineares. Também foram testados HistGradientBoosting (0,6047), Extra-Trees (0,6006) e MLP — rede neural (0,593), sem superar os três *gradient boosters* principais.

7 Otimização de Hiperparâmetros

A otimização foi conduzida com **Optuna** [5], que utiliza busca bayesiana via *Tree-structured Parzen Estimator* (TPE). Cada modelo foi avaliado com um *split* 80/20 estratificado, com 50 *trials* por modelo. Os melhores hiperparâmetros encontrados são apresentados na Tabela 2.

Tabela 2: Hiperparâmetros otimizados via Optuna (50 trials, split 80/20)

Modelo	Hiperparâmetros
LightGBM	n_estimators=1026, max_depth=5, num_leaves=65, learning_rate=0,027, min_child_samples=69, subsample=0,87, colsample_bytree=0,90
XGBoost	n_estimators=310, max_depth=4, learning_rate=0,049, min_child_weight=2, subsample=0,65, colsample_bytree=0,60
CatBoost	iterations=400, depth=7, learning_rate=0,088, l2_leaf_reg=3,19, bagging_temperature=0,92

Foram testadas também buscas mais longas: 150 e 300 *trials* com 5-fold CV completa. Curiosamente, os parâmetros encontrados com a busca mais rápida (50 *trials*, *split* simples) **generalizaram melhor** no Kaggle do que os das buscas mais profundas (Tabela 3). Isso ocorre porque buscas muito longas encontram configurações que exploram idiosincrasias específicas dos folds — uma forma sutil de overfitting ao processo de validação, conforme discutido por Hastie et al. [9].

Tabela 3: Impacto do número de trials na generalização

Configuração Optuna	CV	Kaggle	Obs.
50 trials, split 80/20	0,607	0,6040	Melhor
150 trials, 5-fold CV	0,610	0,6016	
300 trials, 5-fold CV	0,611	0,5988	Pior

8 Ensemble e Submissão

8.1 Voting Ensemble

As previsões dos três modelos otimizados foram combinadas por *soft voting* (média das probabilidades). A previsão final é “inadimplente” se a probabilidade média exceder 0,50.

8.2 Blend multi-seed

Para reduzir a variância intrínseca dos modelos, cada algoritmo foi treinado com 5 sementes aleatórias diferentes, totalizando $3 \times 5 = 15$ modelos. A probabilidade final é a média das 15 probabilidades individuais, reduzindo a variância da previsão.

8.3 Abordagens de ensemble testadas e descartadas

- **Adição de HistGradientBoosting** ao ensemble (4 modelos): piorou de 0,6040 para 0,6004 no Kaggle. Modelos muito similares não adicionam diversidade útil.
- **Modelos fortemente regularizados** (árvores rasas, alta regularização L1/L2): CV de 0,609, Kaggle 0,600.
- **Variação de *threshold*** (0,48 a 0,52): o padrão 0,50 obteve o melhor resultado.

9 Resultados

A Tabela 4 resume todas as submissões realizadas ao Kaggle e seus respectivos scores.

Tabela 4: Resumo das principais submissões ao Kaggle

Descrição	CV	Kaggle
Blend 3×5, Optuna 50t, thr=0,50	0,607	0,6040
Blend 3×5, Optuna 50t, thr=0,51	—	0,6040
Blend 3×5, Optuna 150t (5-fold)	0,610	0,6016
Blend 4×5 (+HistGBM)	—	0,6004
Blend regularizado	0,609	0,6000
Blend 3×5, Optuna 300t (5-fold)	0,611	0,5988
TE expandido (<i>leakage</i>)	0,636	0,5888

O melhor score obtido foi **0,6040**. A principal limitação está na baixa qualidade preditiva das *features*: a maior correlação individual é de apenas 0,12.

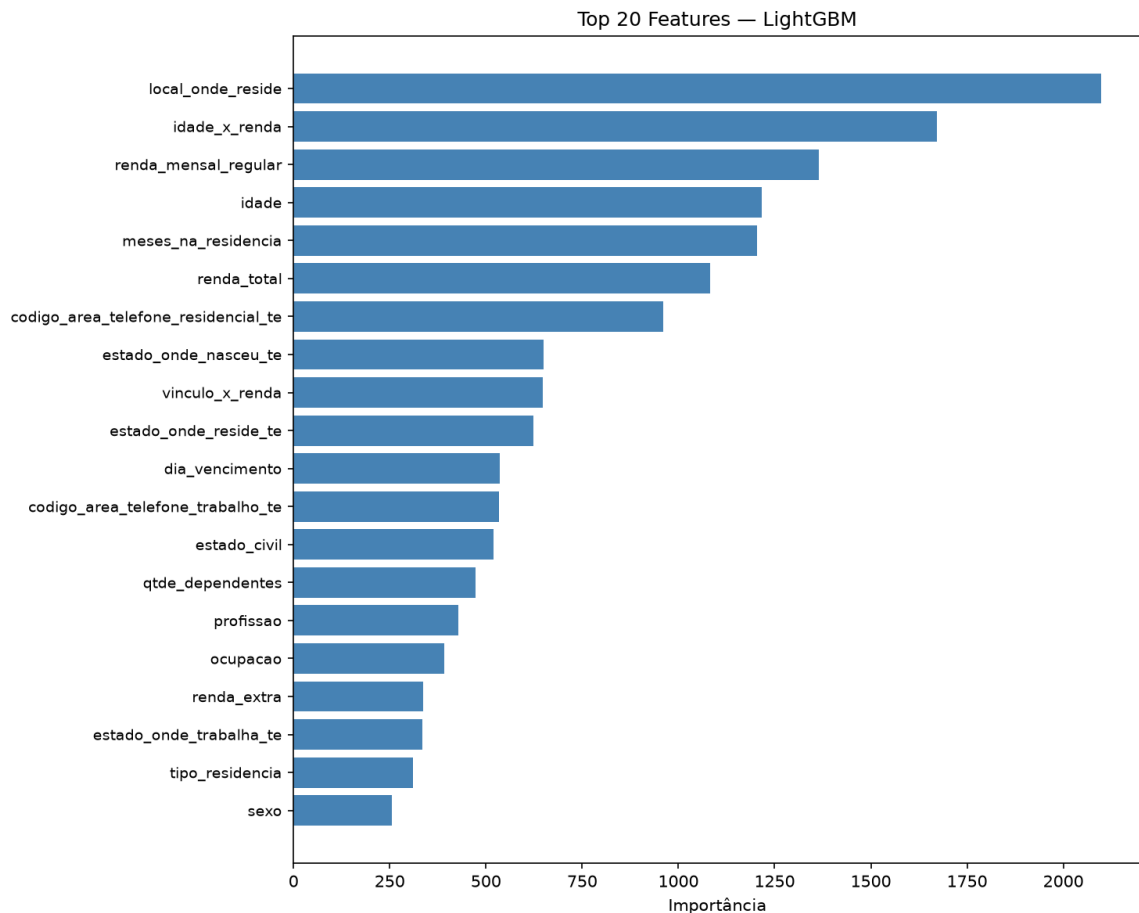
10 Importância das *Features*

A análise de importância do LightGBM (baseada no número de *splits* em que cada variável é utilizada) é apresentada na Tabela 5 e na Figura 2.

Tabela 5: Top 10 *features* por importância (LightGBM)

#	Feature	Importância
1	local_onde_reside	alta
2	idade	alta
3	valor_patrimonio_pessoal	alta
4	renda_mensal_regular	alta
5	idade_x_renda (engenharia)	média-alta
6	estado_onde_reside_te (target enc.)	média-alta
7	razao_patrimonio_renda (engenharia)	média
8	estado_onde_nasceu_te (target enc.)	média
9	meses_na_residencia	média
10	renda_total (engenharia)	média

Destaca-se que 3 das 10 variáveis mais importantes foram criadas na etapa de engenharia de *features* (*idade_x_renda*, *razao_patrimonio_renda*, *renda_total*), e 2 são derivadas de *target encoding* — validando a utilidade dessas etapas.

**Figura 2:** Top 20 *features* por importância no LightGBM otimizado.

11 Conclusões

1. **Gradient boosting** [6] (LightGBM, XGBoost, CatBoost) superou consistentemente modelos lineares e redes neurais em dados tabulares com sinal fraco.
2. **Ensembles** por média de probabilidades e blend multi-seed melhoraram a estabilidade das previsões.
3. **Target encoding** [8] é poderoso para categóricas de alta cardinalidade, mas deve ser restrito a variáveis onde demonstra consistência entre validação e teste real. O uso indiscriminado causa *data leakage*, como verificado experimentalmente.
4. **Mais otimização nem sempre é melhor:** buscas mais longas de hiperparâmetros geraram overfitting ao processo de validação, resultando em scores piores no Kaggle. Isso reforça a importância de validações simples e robustas [9].
5. **O teto de desempenho** é determinado pela qualidade das *features*, não pela sofisticação dos modelos. Múltiplas abordagens diferentes convergiram para a mesma faixa de 0,60–0,61, indicando que o sinal preditivo disponível nos dados é intrinsecamente limitado.

Referências

- [1] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- [2] Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems (NeurIPS)*, v. 30, 2017.
- [3] Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, p. 785–794, 2016.
- [4] Prokhorenkova, L. et al. CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems (NeurIPS)*, v. 31, 2018.
- [5] Akiba, T. et al. Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD*, p. 2623–2631, 2019.
- [6] Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, v. 29, n. 5, p. 1189–1232, 2001.
- [7] Breiman, L. Random Forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.
- [8] Micci-Barreca, D. A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems. *ACM SIGKDD Explorations Newsletter*, v. 3, n. 1, p. 27–32, 2001.
- [9] Hastie, T.; Tibshirani, R.; Friedman, J. *The Elements of Statistical Learning*. 2. ed. New York: Springer, 2009.
- [10] McKinney, W. Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, p. 56–61, 2010.